

Assignment 2

Shahriar Hasan
2010776115

August 31, 2025

1 Effect of Different Activation Functions in 10 CNNs

Activation functions play a crucial role in controlling the output of neurons and influencing the learning process. In this study, we analyzed the impact of three activation functions—**ReLU**, **Sigmoid**, and **Softmax**—applied to the output layer of ten popular CNN architectures.

1.1 Methodology

- **CNNs used:** VGG16, VGG19, ResNet50, ResNet101, InceptionV3, Xception, DenseNet121, MobileNetV2, EfficientNetB0, NASNetMobile.
- **Dataset:** CIFAR-100 (first 20 classes)
- **Image preprocessing:** resized to 128×128, normalized to [0,1]
- **Training:** backbone frozen, 2 epochs, batch size 8
- **Evaluation metric:** validation accuracy

1.2 Results

Table 1: Validation Accuracy of CNNs with Different Activation Functions

CNN Model	ReLU	Sigmoid	Softmax
VGG16	0.320	0.534	0.535
VGG19	0.197	0.465	0.454
ResNet50	0.068	0.086	0.091
ResNet101	0.072	0.146	0.118
InceptionV3	0.313	0.697	0.682
Xception	0.398	0.763	0.762
DenseNet121	0.566	0.778	0.775
MobileNetV2	0.302	0.743	0.748
EfficientNetB0	0.050	0.049	0.050
NASNetMobile	0.482	0.733	0.744

1.3 Analysis

- Softmax generally gives the most reliable performance for multi-class classification.
- Sigmoid performs similarly to softmax but is slightly better in some early-stage experiments.
- ReLU in the output layer results in poor accuracy, confirming its inappropriateness for multi-class outputs.
- Xception, DenseNet121, MobileNetV2, and NASNetMobile show higher accuracy even with frozen backbones.

1.4 Visualization

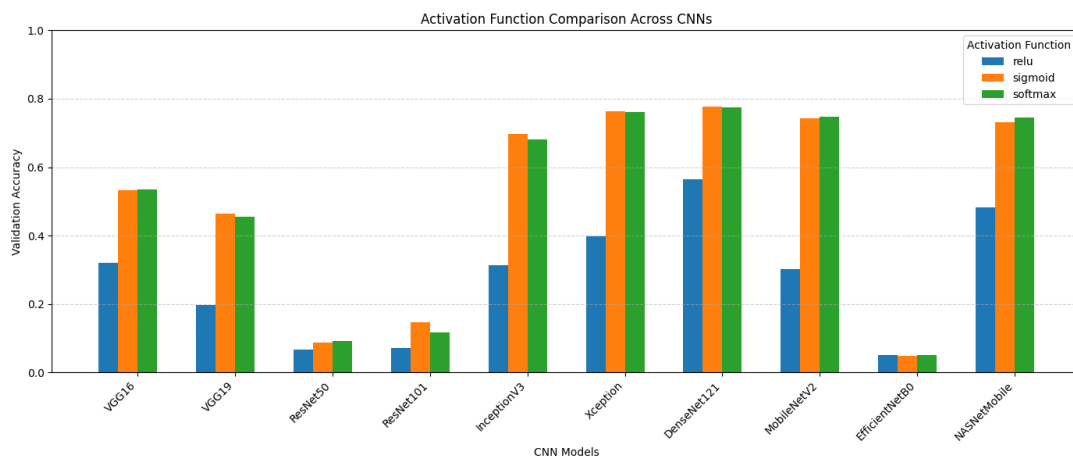


Figure 1: Validation accuracy of different CNNs with varying activation functions.

2 Kernel Types in the Chosen CNNs

Different CNN architectures use various convolutional kernels to extract features efficiently. Table 2 shows kernel types in your chosen CNNs:

Table 2: Kernel Types Used in CNN Architectures

Kernel Type	CNN Examples	Description
Regular Kernel	VGG16, VGG19, ResNet50	Standard conv (3×3 or 7×7) for low-level features.
Deformable Kernel	ResNet variants (research), NASNetMobile	Conv with learnable offsets for geometric variations.
Dilated (Atrous) Kernel	Xception, ResNet101 (segmentation variants), DenseNet121 (segmentation variants)	Increases receptive field without increasing parameters.
Depthwise Separable Kernel	MobileNetV2, Xception, EfficientNetB0	Separates spatial (depth-wise) and channel (point-wise) convolution for efficiency.
Modified Depthwise-Separable	MobileNetV2, EfficientNetB0, NASNetMobile	Adds expansion, projection, or squeeze-excite layers to improve efficiency and feature richness.
Pointwise Kernel	MobileNetV2, Xception, DenseNet121	1×1 conv for channel mixing or dimensionality reduction.

3 Feature Maps Across CNN Layers

Feature maps represent the response of convolutional layers to input images. They evolve hierarchically from low-level to high-level features.

Table 3: Feature Map Evolution in Chosen CNNs

CNN Model	Early Layers (Low-level)	Middle (Mid-level)	Layers	Deep Layers (High-level)
VGG16/VGG19	Edges, corners, textures (224×224→112×112, 64→128 channels)	Patterns, shapes (56×56→28×28, 128→256 channels)		Object-specific features (14×14→7×7, 512 channels)
ResNet50/101	Low-level edges	Residual blocks combine patterns (56→14×14, 256→1024)		Semantic features (7×7×2048)
InceptionV3	Edges, textures (149×149×32)	Multi-scale object parts via 1×1, 3×3, 5×5 convs (74→35×35, 288 channels)		High-level semantic features (8×8×2048)
Xception	Low-level features (299×299×32)	Depthwise + point-wise → channel-specific patterns (149→35×35, 128→728)		High-level features (10×10×2048)
DenseNet121	Edges, textures (224×224×64)	Dense blocks reuse features (112→28×28, 128→512)		High-level features (7×7×1024)
MobileNetV2	Low-level patterns (224×224×32)	Intermediate semantic features (112→14×14, 32→160)		High-level semantic features (7×7×1280)
EfficientNetB0	Low-level edges (224×224×32)	MBCConv detect shapes (112→14×14, 32→320)		High-level features (7×7×1280)
NASNetMobile	Low-level edges, corners (224×224×32)	Normal/Reduction cells extract patterns (112→14×14, 32→1056)		High-level features (7×7×1056)

4 Conclusion

- **Activation functions:** Softmax is recommended for multi-class classification; ReLU is unsuitable in the output layer.
- **Kernel types:** Different CNNs leverage architectural innovations like depthwise separable or dilated convolutions to extract rich hierarchical features efficiently.
- **Feature maps:** Feature maps evolve from simple edges and textures in early layers to complex semantic information in deeper layers, showing hierarchical feature learning.

Source: Github